

AWS SITE-TO-SITE VPN

IMPLEMENTATION DOCUMENTATION

Using AWS CLI + strongSwan IPsec VPN

Project Type	Cloud Networking / VPN Implementation
Technology Stack	AWS VPC, EC2, VGW, CGW, strongSwan, IPsec
OnPrem CIDR	10.10.0.0/16
AWS Cloud CIDR	10.20.0.0/16
VPN Protocol	IKEv1 / IPsec / AES128-SHA1
Document Version	1.0 — Production Ready

CONFIDENTIAL — INTERNAL USE ONLY

1. Introduction and Project Overview

1.1 What Is This Project?

This document provides a complete, step-by-step guide to implementing an AWS Site-to-Site VPN using two Amazon VPCs — one simulating an on-premises corporate data center and the other representing the AWS Cloud environment. The communication between the two private networks is secured using an IPsec VPN tunnel.

This is one of the most critical networking patterns in enterprise cloud architecture. Every large organization that migrates workloads to the cloud requires a secure, private tunnel between their existing data center and their cloud environment. This project replicates that exact scenario.

1.2 Objective

Goal	Description
Primary Goal	Establish encrypted communication between an On-Premises VPC and an AWS Cloud VPC
Protocol	IPsec VPN using IKEv1, AES-128, SHA-1 encryption
Software VPN Gateway	strongSwan installed on Amazon Linux 2 EC2 instance
AWS Components	Virtual Private Gateway (VGW) and Customer Gateway (CGW)
End Result	Private EC2 (10.10.2.47) can ping Private EC2 (10.20.1.177) through tunnel

1.3 Architecture Summary

High-Level Architecture
OnPrem-VPC (10.10.0.0/16) - Public Subnet 10.10.1.0/24 → VPN Appliance EC2 (strongSwan + Elastic IP: 13.126.73.216) - Private Subnet 10.10.2.0/24 → OnPrem Private Instance (10.10.2.47)
AWS-Cloud-VPC (10.20.0.0/16) - Private Subnet 10.20.1.0/24 → AWS Cloud Instance (10.20.1.177) - Virtual Private Gateway (vgw-028a9ed0c1722a0f8)
VPN Tunnel: Elastic IP 13.126.73.216 ↔ AWS Tunnel Endpoint 15.207.7.27

The VPN Appliance EC2 instance acts as a software router and firewall. It has a public Elastic IP address visible to the internet, and it forwards all encrypted VPN traffic between the private subnets of both VPCs.

2. Core Networking Concepts

2.1 What Is a Site-to-Site VPN?

A Site-to-Site VPN is a type of connection that links two separate private networks together securely over the public internet. Think of it as a private, encrypted pipe that runs through the internet — data traveling inside this pipe is fully encrypted, so even if someone intercepts the traffic, they cannot read it. In a traditional enterprise setup, a physical router (like a Cisco or Juniper device) sits at the edge of the corporate network and creates this tunnel to AWS. In our project, we simulate that physical router using a software-based VPN called strongSwan, running on an Amazon Linux 2 EC2 instance.

2.2 What Is IPSec?

IPSec (Internet Protocol Security) is a suite of protocols designed to provide security for IP communications. It works by authenticating and encrypting each IP packet within a communication session.

IPSec has two phases of negotiation:

Phase 1 (IKE - Internet Key Exchange):

- Establishes a secure, authenticated channel between the two VPN endpoints
- Negotiates the encryption algorithm (AES-128), authentication (SHA-1), and DH group (Group 2)
- Creates the ISAKMP Security Association (SA) — a shared secret channel
- This is like two people agreeing on a secret language before they start talking

Phase 2 (IPSec SA):

- Uses the channel established in Phase 1 to create the actual data tunnel
- Creates two unidirectional SAs — one for sending, one for receiving
- This is where actual traffic encryption and decryption happens
- All your data (ping packets, SSH sessions, HTTP requests) flows through here

2.3 What Is IKE?

IKE (Internet Key Exchange) is the protocol used to set up IPSec Security Associations. In our implementation, we use IKEv1. The IKE process works in two modes:

- Main Mode (6 messages): Used when both peers have static IP addresses. Our setup uses this — both the Elastic IP and the AWS Tunnel Endpoint are static.
- Aggressive Mode (3 messages): Faster but less secure. Used when one side has a dynamic IP.

2.4 What Is strongSwan?

strongSwan is an open-source IPSec-based VPN solution for Linux. It is a complete IKE daemon — meaning it handles the complete IPSec tunnel lifecycle: negotiation, establishment, maintenance, and re-keying. In our project, strongSwan runs on the VPN Appliance EC2 instance and handles all VPN communication.

2.5 Virtual Private Gateway (VGW) vs Customer Gateway (CGW)

Component	What It Is	In This Project
Virtual Private Gateway (VGW)	AWS-managed VPN concentrator attached to the AWS VPC	vgw-028a9ed0c1722a0f8 attached to AWS-Cloud-VPC

Component	What It Is	In This Project
Customer Gateway (CGW)	AWS resource representing your on-premises VPN device	cgw-0343d289f5518124a pointing to Elastic IP 13.126.73.216
VPN Connection	The actual tunnel binding VGW and CGW together	vpn-0c196248dd7221d3b

3. Step-by-Step Implementation

PART A: Building the On-Premises VPC

We first build the entire 'On-Premises' side of the network. This represents a corporate data center with its own private IP range, internet access, and internal private servers.

Step 1 — Create the OnPrem VPC

Step Objective

Create the foundational virtual network that will represent the On-Premises corporate data center. A VPC (Virtual Private Cloud) is an isolated network within AWS where you control IP addressing, routing, and security.

```
# Create the OnPrem VPC with CIDR block 10.10.0.0/16
aws ec2 create-vpc \
  --cidr-block 10.10.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=OnPrem-VPC}]'

# Expected Output:
# {
#   "Vpc": {
#     "VpcId": "vpc-xxxxxxxxxxxxxxxx",
#     "CidrBlock": "10.10.0.0/16",
#     "State": "available"
#   }
# }
```

Why 10.10.0.0/16? A /16 gives us 65,536 IP addresses and allows us to create multiple subnets. The 10.x.x.x range is a private address space (RFC 1918) that will never conflict with public internet IPs.

Step 2 — Create and Attach Internet Gateway

Step Objective

An Internet Gateway (IGW) is required to allow the VPN Appliance EC2 instance in the public subnet to send and receive traffic from the internet. Without an IGW, the VPN tunnel cannot be established because the Elastic IP cannot be reached from outside.

```
# Create the Internet Gateway
aws ec2 create-internet-gateway \
  --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=OnPrem-IGW}]'

# Attach the IGW to the OnPrem VPC (replace vpc-xxx with your actual VPC ID)
aws ec2 attach-internet-gateway \
  --internet-gateway-id igw-xxxxxxxxxxxxxxxx \
  --vpc-id vpc-xxxxxxxxxxxxxxxx
```

Think of the Internet Gateway as the front door of your building. Without it, nobody from the outside world can reach your VPN appliance, and the VPN appliance cannot reach AWS either.

Step 3 — Create Public and Private Subnets (OnPrem)

Step Objective

Create two subnets inside the OnPrem VPC: one public (for the VPN appliance that needs internet access) and one private (for internal servers that should never be directly internet-accessible).

```
# Create the Public Subnet (VPN Appliance will live here)
aws ec2 create-subnet \
```

```
--vpc-id vpc-xxxxxxxxxxxxxxxx \
--cidr-block 10.10.1.0/24 \
--availability-zone ap-south-1a \
--tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=OnPrem-Public-Subnet}]'

# Create the Private Subnet (OnPrem Private Instance will live here)
aws ec2 create-subnet \
--vpc-id vpc-xxxxxxxxxxxxxxxx \
--cidr-block 10.10.2.0/24 \
--availability-zone ap-south-1a \
--tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=OnPrem-Private-Subnet}]'
```

Public subnet (10.10.1.0/24): Has a route to the IGW, so instances can access the internet. The VPN Appliance lives here with an Elastic IP. Private subnet (10.10.2.0/24): No direct internet route. Internal servers live here. All outbound traffic for these servers is routed through the VPN Appliance.

Step 4 — Create and Associate Route Tables (OnPrem)

Step Objective

Route tables control where network traffic is directed. We need a public route table (with internet access via IGW) and a private route table (where traffic to AWS goes through the VPN Appliance).

```
# Create Public Route Table
aws ec2 create-route-table \
--vpc-id vpc-xxxxxxxxxxxxxxxx \
--tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=OnPrem-Public-RT}]'

# Add route to Internet Gateway (default route for internet traffic)
aws ec2 create-route \
--route-table-id rtb-public-xxxxxxxx \
--destination-cidr-block 0.0.0.0/0 \
--gateway-id igw-xxxxxxxxxxxxxxxx

# Associate Public Subnet with Public Route Table
aws ec2 associate-route-table \
--route-table-id rtb-public-xxxxxxxx \
--subnet-id subnet-public-xxxxxxxx

# Create Private Route Table
aws ec2 create-route-table \
--vpc-id vpc-xxxxxxxxxxxxxxxx \
--tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=OnPrem-Private-RT}]'

# Route to AWS Cloud (10.20.0.0/16) via VPN Appliance EC2
# NOTE: Add this AFTER launching the VPN Appliance (need its instance ID)
aws ec2 create-route \
--route-table-id rtb-private-xxxxxxxx \
--destination-cidr-block 10.20.0.0/16 \
--instance-id i-xxxxxxxxxxxxxxxx

# Associate Private Subnet with Private Route Table
aws ec2 associate-route-table \
--route-table-id rtb-private-xxxxxxxx \
--subnet-id subnet-private-xxxxxxxx
```

This is the key routing decision: when the OnPrem private instance sends a packet to 10.20.x.x (the AWS Cloud network), the route table says 'send it to the VPN Appliance'. The VPN Appliance then encrypts it and sends it through the IPsec tunnel.

PART B: Building the AWS Cloud VPC

Step 5 — Create the AWS Cloud VPC and Private Subnet

Step Objective

Create the second VPC that represents the AWS Cloud environment. This VPC has only a private subnet since it should only be reachable through the VPN tunnel, not the internet.

```
# Create the AWS Cloud VPC
aws ec2 create-vpc \
  --cidr-block 10.20.0.0/16 \
  --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=AWS-Cloud-VPC}]'

# Create the Private Subnet inside AWS Cloud VPC
aws ec2 create-subnet \
  --vpc-id vpc-aws-cloud-xxxxxxxx \
  --cidr-block 10.20.1.0/24 \
  --availability-zone ap-south-1b \
  --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=AWS-Private-Subnet}]'
```

PART C: Security Groups

Step 6 — Create Security Groups

Step Objective

Security Groups act as virtual firewalls around EC2 instances. We need specific rules: the VPN Appliance must allow UDP 500 and UDP 4500 (IPSec/IKE ports), and both private instances need to allow ICMP (ping) traffic from each other's subnets.

```
# Security Group for VPN Appliance (OnPrem)
aws ec2 create-security-group \
  --group-name VPN-Appliance-SG \
  --description 'Security group for VPN Appliance' \
  --vpc-id vpc-onprem-xxxxxxxx

# Allow SSH from your management IP
aws ec2 authorize-security-group-ingress \
  --group-id sg-vpnappliance-xxxxx \
  --protocol tcp --port 22 --cidr YOUR_IP/32

# Allow IKE (UDP 500) from AWS Tunnel Endpoint
aws ec2 authorize-security-group-ingress \
  --group-id sg-vpnappliance-xxxxx \
  --protocol udp --port 500 --cidr 0.0.0.0/0

# Allow NAT-Traversal (UDP 4500) for IPSec over NAT
aws ec2 authorize-security-group-ingress \
  --group-id sg-vpnappliance-xxxxx \
  --protocol udp --port 4500 --cidr 0.0.0.0/0

# Allow all traffic from OnPrem private subnet (for packet forwarding)
aws ec2 authorize-security-group-ingress \
  --group-id sg-vpnappliance-xxxxx \
  --protocol -1 --cidr 10.10.0.0/16

# Security Group for OnPrem Private Instance
aws ec2 create-security-group \
  --group-name OnPrem-Private-SG \
  --description 'OnPrem private instance SG' \
  --vpc-id vpc-onprem-xxxxxxxx
```

```
aws ec2 authorize-security-group-ingress \
  --group-id sg-onprempriate-xxxxx \
  --protocol icmp --port -1 --cidr 10.20.0.0/16

# Security Group for AWS Cloud Private Instance
aws ec2 create-security-group \
  --group-name AWS-Cloud-Private-SG \
  --description 'AWS Cloud private instance SG' \
  --vpc-id vpc-aws-cloud-xxxxxxxxx

aws ec2 authorize-security-group-ingress \
  --group-id sg-awscldprivate-xxxxx \
  --protocol icmp --port -1 --cidr 10.10.0.0/16

aws ec2 authorize-security-group-ingress \
  --group-id sg-awscldprivate-xxxxx \
  --protocol tcp --port 22 --cidr 10.10.0.0/16
```

Why UDP 500 and UDP 4500?

UDP 500: Used by IKE (Internet Key Exchange) for Phase 1 negotiation. When two VPN peers first connect, they use this port to agree on encryption keys and algorithms.

UDP 4500: Used for NAT-Traversal. When one VPN endpoint is behind a NAT device, IKE traffic is encapsulated in UDP 4500 to pass through the NAT. AWS VGW always uses this even for non-NAT scenarios.

PART D: Launching EC2 Instances

Step 7 — Launch the VPN Appliance EC2

Step Objective

Launch an Amazon Linux 2 EC2 instance in the OnPrem Public Subnet. This instance will run strongSwan and serve as the software VPN gateway. It needs to be in the public subnet to have internet reachability.

```
# Launch VPN Appliance in OnPrem Public Subnet
aws ec2 run-instances \
  --image-id ami-0f58b397bc5c1f2e8 \
  --instance-type t2.micro \
  --key-name your-key-pair \
  --subnet-id subnet-onprem-public-xxxxx \
  --security-group-ids sg-vpnappliance-xxxxx \
  --associate-public-ip-address \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=VPN-Appliance}]'
```

Step 8 — Disable Source/Destination Check (CRITICAL)

Step Objective — WHY This Is Critical

By default, every EC2 instance checks that traffic it receives is either FROM or TO its own IP address. If the traffic fails this check, the instance drops it.

The VPN Appliance needs to FORWARD traffic — it receives packets from 10.10.2.47 (not its own IP) and forwards them. Without disabling this check, the instance drops all forwarded traffic and VPN routing fails completely.

```
# Disable Source/Destination Check on VPN Appliance
aws ec2 modify-instance-attribute \
  --instance-id i-xxxxxxxxxxxxxxxxx \
  --no-source-dest-check

# Verify the check is disabled
```

```
aws ec2 describe-instance-attribute \
  --instance-id i-xxxxxxxxxxxxxxxx \
  --attribute sourceDestCheck

# Expected output:
# { "SourceDestCheck": { "Value": false } }
```

Step 9 — Allocate and Associate Elastic IP

Step Objective

A standard public IP assigned by AWS changes every time the instance is stopped and started. An Elastic IP is a static public IP that stays the same permanently. The Customer Gateway is configured with this IP, so it must never change.

```
# Allocate a new Elastic IP
aws ec2 allocate-address --domain vpc

# Note the AllocationId and PublicIp from the output
# PublicIp: 13.126.73.216 (your actual Elastic IP)

# Associate Elastic IP with VPN Appliance
aws ec2 associate-address \
  --instance-id i-xxxxxxxxxxxxxxxx \
  --allocation-id eipalloc-xxxxxxxxxxxxxxxx
```

Step 10 — Launch Private Instances

```
# Launch OnPrem Private Instance (will test VPN from this instance)
aws ec2 run-instances \
  --image-id ami-0f58b397bc5c1f2e8 \
  --instance-type t2.micro \
  --key-name your-key-pair \
  --subnet-id subnet-onprem-private-xxxxx \
  --security-group-ids sg-onprempriate-xxxxx \
  --no-associate-public-ip-address \
  --private-ip-address 10.10.2.47 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=OnPrem-Private}]'

# Launch AWS Cloud Private Instance
aws ec2 run-instances \
  --image-id ami-0f58b397bc5c1f2e8 \
  --instance-type t2.micro \
  --key-name your-key-pair \
  --subnet-id subnet-aws-private-xxxxx \
  --security-group-ids sg-awscloudprivate-xxxxx \
  --no-associate-public-ip-address \
  --private-ip-address 10.20.1.177 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=AWS-Cloud-Private}]'
```

4. AWS VPN Components Setup

PART E: Virtual Private Gateway, Customer Gateway, and VPN Connection

Step 11 — Create Virtual Private Gateway (VGW)

Step Objective

The Virtual Private Gateway is the AWS-side VPN concentrator. It is a managed, highly available service that terminates VPN connections from the customer side. Think of it as AWS's router that connects your VPN tunnel to your VPC.

```
# Create the Virtual Private Gateway
aws ec2 create-vpn-gateway \
  --type ipsec.1 \
  --tag-specifications 'ResourceType=vpn-gateway,Tags=[{Key=Name,Value=AWS-Cloud-VGW}]'

# Output will include the VGW ID:
# VpnGatewayId: vgw-028a9ed0c1722a0f8

# Attach VGW to AWS Cloud VPC
aws ec2 attach-vpn-gateway \
  --vpn-gateway-id vgw-028a9ed0c1722a0f8 \
  --vpc-id vpc-aws-cloud-xxxxxxxxx

# Verify attachment state
aws ec2 describe-vpn-gateways \
  --vpn-gateway-ids vgw-028a9ed0c1722a0f8
# Look for: "State": "attached"
```

Step 12 — Create Customer Gateway (CGW)

Step Objective

The Customer Gateway is an AWS resource that represents your on-premises VPN device. It stores the public IP address of your VPN appliance (the Elastic IP: 13.126.73.216) so that AWS knows where to send VPN traffic.

```
# Create Customer Gateway pointing to VPN Appliance's Elastic IP
aws ec2 create-customer-gateway \
  --type ipsec.1 \
  --public-ip 13.126.73.216 \
  --bgp-asn 65000 \
  --tag-specifications 'ResourceType=customer-gateway,Tags=[{Key=Name,Value=OnPrem-CGW}]'

# Output:
# CustomerGatewayId: cgw-0343d289f5518124a
```

BGP ASN 65000 is a standard private AS number. Even though we are not using dynamic BGP routing in this project (we use static routes), AWS requires a BGP ASN value when creating a Customer Gateway.

Step 13 — Create Site-to-Site VPN Connection

Step Objective

The VPN Connection ties the Virtual Private Gateway (AWS side) and the Customer Gateway (OnPrem side) together. AWS automatically creates TWO tunnels for redundancy — if one fails, traffic automatically fails over to the other.

```
# Create the VPN Connection with static routing
aws ec2 create-vpn-connection \
  --type ipsec.1 \
  --customer-gateway-id cgw-0343d289f5518124a \
  --vpn-gateway-id vgw-028a9ed0c1722a0f8 \
  --options '{"StaticRoutesOnly":true}' \
  --tag-specifications 'ResourceType=vpn-connection,Tags=[{Key=Name,Value=OnPrem-to-AWS-VPN}]'

# VPN Connection ID: vpn-0c196248dd7221d3b

# Add static route telling AWS which CIDRs are behind the Customer Gateway
aws ec2 create-vpn-connection-route \
  --vpn-connection-id vpn-0c196248dd7221d3b \
  --destination-cidr-block 10.10.0.0/16
```

Step 14 — Download VPN Configuration

Step Objective

After the VPN connection is created, download the configuration file from AWS. This file contains all the critical parameters: tunnel endpoints, pre-shared keys, encryption settings. We use this to configure strongSwan.

```
# Download VPN configuration (Generic vendor, strongSwan format)
aws ec2 get-vpn-connection-device-sample-configuration \
  --vpn-connection-id vpn-0c196248dd7221d3b \
  --vpn-connection-device-type-id generic \
  --internet-key-exchange-version ikev1 \
  --output text > vpn-config.txt

# Review the downloaded configuration
cat vpn-config.txt

# Key values to extract from the config:
# Tunnel 2 Endpoint (Outside IP): 15.207.7.27
# Pre-Shared Key: dq_vUvBgbDEd0_vY.EAwQtYFDh pz7Dxm
# IKE Version: IKEv1
# Encryption: AES-128
# Authentication: SHA1
```

5. strongSwan Configuration

5.1 Install strongSwan on VPN Appliance

SSH into the VPN Appliance EC2 instance (the one with Elastic IP 13.126.73.216). All the following commands are run on this instance.

```
# SSH into VPN Appliance
ssh -i your-key.pem ec2-user@13.126.73.216

# Update the system
sudo yum update -y

# Install strongSwan from EPEL repository
sudo amazon-linux-extras install epel -y
sudo yum install strongswan -y

# Verify installation
strongswan version
# Expected: strongSwan 5.x.x

# Check if ipsec binary is available
which ipsec
# Expected: /usr/sbin/ipsec
```

5.2 Enable IP Forwarding

Step Objective — CRITICAL

By default, Linux kernels do NOT forward packets between network interfaces. IP Forwarding is the kernel-level setting that allows the machine to act as a router — receiving packets on one interface and forwarding them out another.

Without IP Forwarding enabled, the VPN Appliance can receive encrypted packets from the tunnel but cannot forward the decrypted data to the internal private subnet. The ping will fail silently.

```
# Enable IP forwarding immediately (without reboot)
sudo sysctl -w net.ipv4.ip_forward=1

# Make it permanent (survives reboots)
echo 'net.ipv4.ip_forward = 1' | sudo tee -a /etc/sysctl.conf

# Also enable forwarding for VPN interfaces
echo 'net.ipv4.conf.all.forwarding = 1' | sudo tee -a /etc/sysctl.conf

# Apply changes
sudo sysctl -p

# Verify
cat /proc/sys/net/ipv4/ip_forward
# Expected output: 1
```

5.3 Configure ipsec.conf

Step Objective

The ipsec.conf file is the main strongSwan configuration file. It defines everything about the VPN tunnel: which IPs are involved, what encryption to use, which subnets are on each side, and how to handle connection failures.

```
sudo nano /etc/strongswan/ipsec.conf
# OR
sudo vim /etc/strongswan/ipsec.conf
```

Paste the following configuration:

```
config setup
    charondebug="ike 1, knl 1, cfg 1"
    uniqueids=no

conn aws-vpn
    auto=start
    type=tunnel
    keyexchange=ikev1
    authby=psk

    left=%defaultroute
    leftid=13.126.73.216
    leftsubnet=10.10.0.0/16

    right=15.207.7.27
    rightsubnet=10.20.0.0/16

    ike=aes128-sha1-modp1024!
    esp=aes128-sha1-modp1024!

    ikelifetime=8h
    lifetime=1h
    margintime=5m

    dpdaction=restart
    dpddelay=10s
    dpdtimeout=30s
```

ipsec.conf Parameter Explanation

Parameter	Value	Explanation
charondebug	ike 1, knl 1, cfg 1	Enable debug logging for IKE negotiation, kernel, and config. Level 1 = minimal but useful.
uniqueids=no	no	Allow multiple connections with same identity. Important for AWS dual-tunnel setup.
auto=start	start	Automatically start and maintain this tunnel when strongSwan starts.
type=tunnel	tunnel	Use Tunnel mode (encrypts entire IP packet). As opposed to Transport mode (only encrypts payload).
keyexchange	ikev1	Use IKEv1. AWS supports both but our downloaded config specifies IKEv1.
authby=psk	psk	Authenticate using Pre-Shared Key (PSK). The key is in ipsec.secrets file.
left	%defaultroute	The local (our) side IP. %defaultroute means auto-detect from the routing table.
leftid	13.126.73.216	Our public identity: the Elastic IP. AWS VGW uses this to identify us.
leftsubnet	10.10.0.0/16	The subnet behind us (on-premises). Traffic from this subnet goes through the tunnel.
right	15.207.7.27	The remote (AWS) side: the VGW tunnel endpoint IP.

Parameter	Value	Explanation
rightsubnet	10.20.0.0/16	The subnet behind AWS. Traffic destined for this goes through the tunnel.
ike=aes128-sha1-modp1024!	AES-128+SHA-1+DH2	Phase 1 cipher suite. '!' means exact match required, no negotiation.
esp=aes128-sha1-modp1024!	AES-128+SHA-1+DH2	Phase 2 cipher suite for actual data encryption.
ikelifetime=8h	28800s	Phase 1 SA lifetime. Matches AWS default of 28800 seconds.
lifetime=1h	3600s	Phase 2 SA (data tunnel) lifetime. Re-keyed every hour.
dpdaction=restart	restart	Dead Peer Detection: if peer is unresponsive, restart the tunnel.

5.4 Configure ipsec.secrets

Step Objective

The ipsec.secrets file stores the Pre-Shared Key (PSK). This key must exactly match what AWS provided in the VPN configuration download. Both sides use this key during IKE Phase 1 to prove their identity to each other.

```
sudo nano /etc/strongswan/ipsec.secrets

# Add this single line:
13.126.73.216 15.207.7.27 : PSK "dq_vUvBgbDEd0_vY.EAwQtYFDhpz7Dxm"

# Secure the file (PSK should not be readable by others)
sudo chmod 600 /etc/strongswan/ipsec.secrets
```

Format: [LocalID] [RemoteID] : PSK "[key]". The LocalID is our Elastic IP, RemoteID is the AWS tunnel endpoint. The PSK provided by AWS in the VPN configuration download must be entered here exactly — even a single character difference will cause the tunnel to fail with an authentication error.

5.5 Start strongSwan and Bring Up the Tunnel

```
# Start the strongSwan service
sudo systemctl start strongswan

# Enable auto-start on boot
sudo systemctl enable strongswan

# Check service status
sudo systemctl status strongswan

# Manually initiate the VPN connection
sudo ipsec up aws-vpn

# Check tunnel status
sudo ipsec status
sudo ipsec statusall

# Expected: you should see ESTABLISHED and INSTALLED
# ESTABLISHED means Phase 1 (IKE) succeeded
# INSTALLED means Phase 2 (IPSec SA) is active and data can flow

# View real-time VPN negotiation logs
sudo journalctl -u strongswan -f
```

```
# OR  
sudo tail -f /var/log/secure
```

6. Route Propagation and Routing Setup

6.1 Enable Route Propagation on AWS Cloud VPC

Step Objective

Route Propagation allows the Virtual Private Gateway to automatically inject routes into the AWS Cloud VPC's route table. When the VPN tunnel is up, the VGW will automatically add a route for 10.10.0.0/16 pointing to itself. Without this, packets destined for the OnPrem network have no route and are dropped.

```
# Enable route propagation on the AWS Cloud VPC route table
aws ec2 enable-vgw-route-propagation \
  --route-table-id rtb-aws-private-xxxxxxx \
  --gateway-id vgw-028a9ed0c1722a0f8

# Verify the propagated route appears
aws ec2 describe-route-tables \
  --route-table-ids rtb-aws-private-xxxxxxx

# Look for a route with:
# DestinationCidrBlock: 10.10.0.0/16
# GatewayId: vgw-028a9ed0c1722a0f8
# Origin: EnableVgwRoutePropagation
```

6.2 Complete Route Table Summary

The following table shows all routes that must be present for end-to-end connectivity:

Route Table	Destination	Target	Purpose
OnPrem-Public-RT	0.0.0.0/0	igw-xxx (Internet Gateway)	VPN Appliance reaches internet
OnPrem-Public-RT	10.10.0.0/16	local	Local VPC traffic
OnPrem-Private-RT	10.20.0.0/16	i-xxx (VPN Appliance)	Route AWS-bound traffic to VPN Appliance
OnPrem-Private-RT	10.10.0.0/16	local	Local VPC traffic
AWS-Cloud-Private-RT	10.10.0.0/16	vgw-028a9ed0c1722a0f8	Auto-propagated by VGW — OnPrem traffic
AWS-Cloud-Private-RT	10.20.0.0/16	local	Local VPC traffic

7. End-to-End Packet Flow Story

This section tells the complete story of a single ICMP ping packet traveling from the OnPrem private instance to the AWS Cloud instance. Understanding this flow deeply will make troubleshooting much easier.

7.1 The Complete Journey of a Ping Packet

Imagine you are sitting at the OnPrem Private Instance (10.10.2.47) and you type: ping 10.20.1.177. Here is exactly what happens:

Stage 1 — Packet Creation at Source (10.10.2.47)

Inside the Private Instance

1. The OS creates an ICMP Echo Request packet:
Source IP: 10.10.2.47
Destination IP: 10.20.1.177
Protocol: ICMP (Type 8 - Echo Request)
2. The OS checks its routing table:
10.20.1.177 does not match 10.10.0.0/16 (local)
The private route table has: 10.20.0.0/16 → VPN Appliance (10.10.1.x)
3. The packet is sent to the VPN Appliance as the next hop.

Stage 2 — VPN Appliance Receives the Packet

Inside the VPN Appliance (10.10.1.x, Elastic IP 13.126.73.216)

1. The packet arrives at the VPN Appliance's eth0 interface.
Note: The packet is DESTINED for 10.20.1.177, NOT for the appliance itself.
Normally Linux would DROP this packet. But Source/Dest Check is disabled and IP Forwarding is enabled, so Linux accepts and processes it.
2. The Linux kernel checks its routing table.
It finds: strongSwan has installed a route — 10.20.0.0/16 goes through the IPSec tunnel.
3. The packet is handed to the strongSwan/IPSec kernel module.

Stage 3 — IPSec Encryption (IKE Phase 1 + Phase 2)

Inside IPSec / strongSwan

PHASE 1 (if tunnel not already up):

1. Charon (strongSwan daemon) sends IKE_SA_INIT to 15.207.7.27 over UDP 500.
2. Both sides negotiate: AES-128 encryption, SHA-1 authentication, DH Group 2.
3. Diffie-Hellman key exchange happens — both sides compute a shared secret WITHOUT transmitting the secret over the wire. This is the mathematical magic of DH.
4. Both sides authenticate using the Pre-Shared Key (PSK).
5. An IKE Security Association (SA) is established. This is Phase 1 complete.

PHASE 2 (IPSec SA):

6. Inside the secure Phase 1 channel, both sides negotiate Phase 2 parameters.

7. Two IPSec SAs are created: one for inbound, one for outbound.
8. Each SA has an SPI (Security Parameter Index) — a 32-bit number identifying the SA.

ENCRYPTION:

9. The original ICMP packet is taken by the IPSec stack.
10. It is encrypted using AES-128 in CBC mode.
11. An HMAC-SHA1 authentication tag is appended.
12. An ESP (Encapsulating Security Payload) header is prepended.
13. A new outer IP header is added:
 - New Source IP: 13.126.73.216 (Elastic IP)
 - New Destination IP: 15.207.7.27 (AWS Tunnel Endpoint)
 - Protocol: ESP (50)
14. The encrypted packet travels over the public internet. Nobody can read it.

Stage 4 — Transit Over Public Internet

The ESP-encapsulated encrypted packet travels from the Elastic IP (13.126.73.216) through the internet to the AWS Tunnel Endpoint (15.207.7.27). To any network observer, this looks like a random stream of encrypted bytes. The original ICMP packet is completely invisible.

Stage 5 — AWS VGW Receives and Decrypts

Inside the AWS Virtual Private Gateway

1. The VGW receives the ESP packet at 15.207.7.27.
2. It looks up the SPI in the ESP header to find the correct Security Association.
3. It decrypts the packet using AES-128 and verifies the HMAC-SHA1 signature.
4. The original ICMP packet is recovered:
 - Source IP: 10.10.2.47
 - Destination IP: 10.20.1.177
5. The VGW injects this packet into the AWS-Cloud-VPC.

Stage 6 — Routing Inside AWS Cloud VPC

Inside AWS-Cloud-VPC Routing

1. The packet is now inside the AWS-Cloud-VPC with destination 10.20.1.177.
2. The AWS Cloud Private Route Table has: 10.20.0.0/16 → local
3. AWS routes the packet directly to the instance with IP 10.20.1.177.
4. The Security Group on the AWS Cloud instance allows ICMP from 10.10.0.0/16.
5. The ICMP Echo Request is delivered to 10.20.1.177.

Stage 7 — Response Packet (Return Journey)

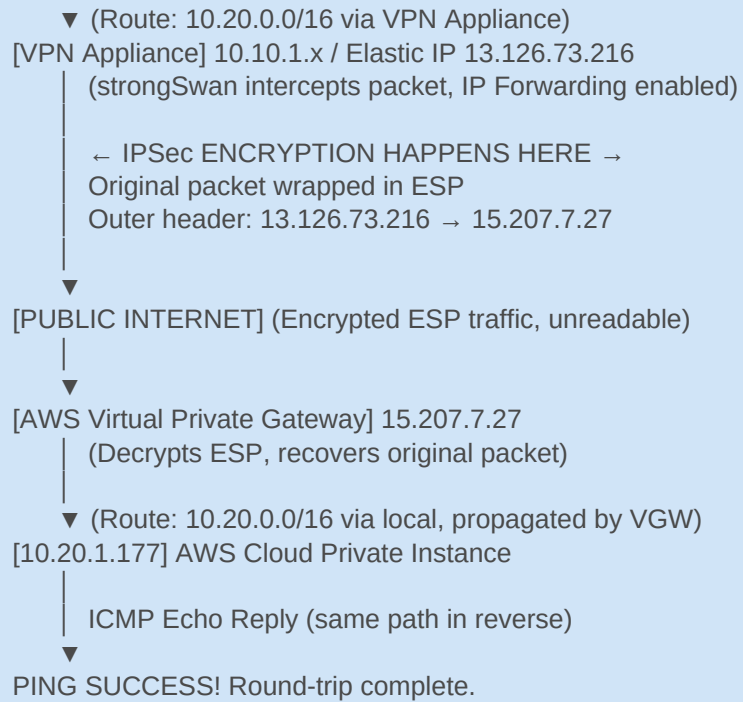
The AWS Cloud instance (10.20.1.177) generates an ICMP Echo Reply. This reply travels the exact same path in reverse: 10.20.1.177 → VGW → Encrypt → Internet → VPN Appliance → Decrypt → 10.10.2.47. The ping succeeds and the round-trip time is measured.

7.2 Visual Summary of Packet Flow

Complete Packet Flow

[10.10.2.47] OnPrem Private Instance

ICMP Echo Request (plaintext, 10.10.2.47 → 10.20.1.177)



8. Verification Steps

8.1 Verify Tunnel Status from VPN Appliance

```
# Connect to VPN Appliance
ssh -i your-key.pem ec2-user@13.126.73.216

# Check IPsec tunnel status
sudo ipsec status

# Expected healthy output:
# Security Associations (1 up, 0 connecting):
#   aws-vpn[1]: ESTABLISHED 5 minutes ago, 10.10.1.x...15.207.7.27
#   aws-vpn{1}: INSTALLED, TUNNEL, reqid 1
#   aws-vpn{1}: 10.10.0.0/16 === 10.20.0.0/16

# Detailed status including all SAs
sudo ipsec statusall

# Check IPsec kernel policies and states
sudo ip xfrm state
sudo ip xfrm policy

# Expected: You should see multiple entries with
# src/dst matching your subnets and protocol esp
```

8.2 Verify from AWS Console

```
# Check VPN Connection status via CLI
aws ec2 describe-vpn-connections \
  --vpn-connection-ids vpn-0c196248dd7221d3b

# Look for:
# State: available
# Tunnel status: UP (at least one of the two tunnels should be UP)

# Check VGW attachment
aws ec2 describe-vpn-gateways \
  --vpn-gateway-ids vgw-028a9ed0c1722a0f8
# VpcAttachments[0].State should be: attached
```

8.3 Test End-to-End Connectivity — The Final Ping Test

Important Note on Access

The OnPrem Private Instance (10.10.2.47) has no public IP. You must:

1. SSH into the VPN Appliance (13.126.73.216) using your key pair
2. From there, SSH into the Private Instance (10.10.2.47) using the same or another key
3. Then ping 10.20.1.177 from the private instance

```
# Step 1: SSH to VPN Appliance
ssh -i your-key.pem ec2-user@13.126.73.216

# Step 2: From VPN Appliance, SSH to Private Instance
ssh -i your-key.pem ec2-user@10.10.2.47

# Step 3: Ping the AWS Cloud Private Instance
ping 10.20.1.177

# Expected successful output:
# PING 10.20.1.177 (10.20.1.177) 56(84) bytes of data.
```

```
# 64 bytes from 10.20.1.177: icmp_seq=1 ttl=253 time=X.XX ms
# 64 bytes from 10.20.1.177: icmp_seq=2 ttl=253 time=X.XX ms

# Verify the route being used
traceroute 10.20.1.177
# Should show only 2-3 hops (private → VPN appliance → AWS target)
```

9. Troubleshooting Guide

Problem: Tunnel is DOWN / Not ESTABLISHED

Symptoms:

- ipsec status shows CONNECTING but never reaches ESTABLISHED
- AWS Console shows Tunnel Status: DOWN

Root Causes:

- Security Group blocking UDP 500/4500
- Pre-Shared Key mismatch
- Encryption algorithm mismatch
- VPN Appliance Source/Dest check still enabled

Solutions:

1. Check SG: `aws ec2 describe-security-groups` (verify UDP 500 and 4500 are open)
2. Verify PSK in `ipsec.secrets` matches exactly what AWS provided
3. Confirm `ike=aes128-sha1-modp1024!` matches Tunnel 2 settings
4. Verify Source/Dest check: `aws ec2 describe-instance-attribute --attribute sourceDestCheck`

Problem: PSK Mismatch

Symptoms:

- strongSwan log shows: 'authentication of X with pre-shared key failed'
- Tunnel goes to CONNECTING then immediately fails

Root Causes:

- Wrong PSK copied from configuration
- Extra whitespace or newline in `ipsec.secrets`

Solutions:

5. Re-download the VPN configuration from AWS console
6. Re-check `ipsec.secrets`: `cat /etc/strongswan/ipsec.secrets`
7. Ensure no extra spaces around the PSK string
8. Re-run: `sudo ipsec reload && sudo ipsec up aws-vpn`

Problem: Ping Fails Even When Tunnel is UP

Symptoms:

- ipsec status shows ESTABLISHED and INSTALLED
- ping still fails from private instance

Root Causes:

- Route table not updated (missing 10.20.0.0/16 route via VPN appliance)
- IP forwarding not enabled on VPN Appliance
- Security Group on destination instance blocks ICMP

Solutions:

9. Check private route table: `aws ec2 describe-route-tables`
10. Verify IP forwarding: `cat /proc/sys/net/ipv4/ip_forward` (must be 1)

11. Check Security Group on AWS Cloud instance allows ICMP from 10.10.0.0/16
12. Add route: `aws ec2 create-route --destination-cidr-block 10.20.0.0/16 --instance-id VPN_APPLIANCE_ID`

Problem: Route Propagation Not Working

Symptoms:

- AWS Cloud instance cannot receive traffic
- Route for 10.10.0.0/16 not visible in AWS Cloud route table

Root Causes:

- Route propagation not enabled on the route table
- VGW not attached to VPC

Solutions:

13. Enable propagation: `aws ec2 enable-vgw-route-propagation --route-table-id RTB_ID --gateway-id VGW_ID`
14. Verify VGW attachment: `aws ec2 describe-vpn-gateways` (look for State: attached)
15. Verify the static route was added to VPN connection: `aws ec2 describe-vpn-connections`

Problem: strongSwan Service Not Starting

Symptoms:

- `systemctl status strongswan` shows failed
- 'ipsec' command not found

Root Causes:

- strongSwan not installed properly
- Configuration file syntax error

Solutions:

16. Reinstall: `sudo yum remove strongswan -y && sudo yum install strongswan -y`
17. Check config syntax: `sudo ipsec --checkconfigfiles`
18. View detailed error: `sudo journalctl -u strongswan --no-pager -n 50`
19. Ensure EPEL repo enabled: `sudo amazon-linux-extras install epel -y`

10. Best Practices and Security Recommendations

10.1 Security Recommendations

Area	Recommendation
Encryption	Use IKEv2 instead of IKEv1 for production. IKEv2 is faster, more secure, and supports MOBIKE.
Algorithms	Upgrade to AES-256-GCM + SHA-256 + DH Group 14 or 20 (ECDH) for stronger encryption.
Pre-Shared Key	Use a strong PSK (32+ random characters). Consider using certificate-based authentication for production.
Key Rotation	Rotate Pre-Shared Keys every 90 days. AWS allows you to change the PSK from the VPN Connection settings.
Monitoring	Enable AWS CloudWatch metrics for VPN connections. Set alarms on TunnelState metric.
Redundancy	Always configure BOTH tunnels provided by AWS for High Availability. If one tunnel's endpoint fails, traffic shifts to the second automatically.
Access Control	Restrict Security Group rules to specific IPs. Never use 0.0.0.0/0 for UDP 500/4500 in production; use the specific AWS tunnel endpoint IPs.
Logging	Enable VPC Flow Logs on both VPCs to capture all traffic for audit and forensics.

10.2 Operational Best Practices

- Always use Elastic IPs for VPN appliances — dynamic IPs break the Customer Gateway configuration every time the instance restarts.
- Tag all resources consistently (Project, Environment, Owner) for cost attribution and management.
- Use AWS Systems Manager Session Manager instead of SSH bastion for accessing private instances — it removes the need for inbound SSH rules.
- Test DPD (Dead Peer Detection) settings carefully. Too aggressive DPD can cause unnecessary tunnel restarts on transient network issues.
- In production, consider AWS Direct Connect instead of or alongside VPN for guaranteed bandwidth and lower latency.

11. Learning Outcomes, Use Cases, and Enterprise Importance

11.1 Key Learning Outcomes

After completing this implementation, you will have hands-on understanding of:

20. How VPCs, subnets, route tables, and internet gateways work together in AWS networking
21. The complete IPsec handshake process — IKE Phase 1 and Phase 2 negotiation
22. How strongSwan operates as a Linux IPsec daemon and how to configure it
23. The role of Virtual Private Gateway and Customer Gateway in AWS VPN architecture
24. How Linux IP forwarding and routing tables enable packet forwarding between networks
25. End-to-end packet encryption flow — how plaintext becomes ciphertext and travels the internet safely
26. How route propagation works and why it is critical for VPN routing in AWS
27. Troubleshooting methodology for VPN connectivity issues

11.2 Real-World Use Cases

Use Case	Description
Cloud Migration	Enterprises migrating workloads to AWS establish VPN connections first, allowing gradual migration while keeping on-premises and cloud systems communicating securely.
Hybrid Cloud Architecture	Organizations run databases on-premises (for compliance or data sovereignty) while running application servers in AWS, with VPN providing the secure link.
Disaster Recovery	Companies replicate data from on-premises to AWS over a VPN. In a disaster scenario, they fail over to AWS while the VPN provides seamless connectivity.
Dev/Test Environments	Development teams in corporate offices access AWS resources in private subnets directly from their workstations without exposing those resources to the internet.
Multi-Cloud Connectivity	Organizations with Azure and AWS deployments use site-to-site VPNs to enable cross-cloud communication for shared services.
Retail/Branch Office	Large retailers connect each store branch to a central AWS cloud infrastructure for POS system integration, inventory, and analytics.

11.3 Enterprise Importance of Site-to-Site VPN

Site-to-Site VPN is one of the foundational services of enterprise cloud adoption. Here is why it matters at scale:

- **Security and Compliance:** Financial institutions (banks, insurance) and healthcare organizations (HIPAA) are required by regulation to encrypt all data in transit. Site-to-Site VPN provides this encryption layer between corporate data centers and cloud environments.
- **Business Continuity:** Most large enterprises do not fully migrate to the cloud overnight. They operate in a hybrid state for years or permanently. VPN is the backbone that keeps these hybrid architectures functioning securely.

- **Cost Efficiency:** Compared to AWS Direct Connect (which requires dedicated leased lines), VPN over the internet is significantly cheaper, making it the preferred choice for smaller workloads or organizations getting started with cloud.
- **Speed of Deployment:** A Site-to-Site VPN can be set up in under an hour (as this guide demonstrates), whereas physical network connections can take weeks or months to provision.
- **Operational Transparency:** Traffic flowing through the VPN tunnel is encrypted but still traceable at the endpoint level, allowing security teams to monitor, audit, and investigate network flows without exposing the actual data.

Appendix — Full Architecture Diagram

The diagram below shows the complete end-to-end architecture of the AWS Site-to-Site VPN implementation, including both VPCs, all networking components, the IPSec tunnel, and the traffic flow between instances.

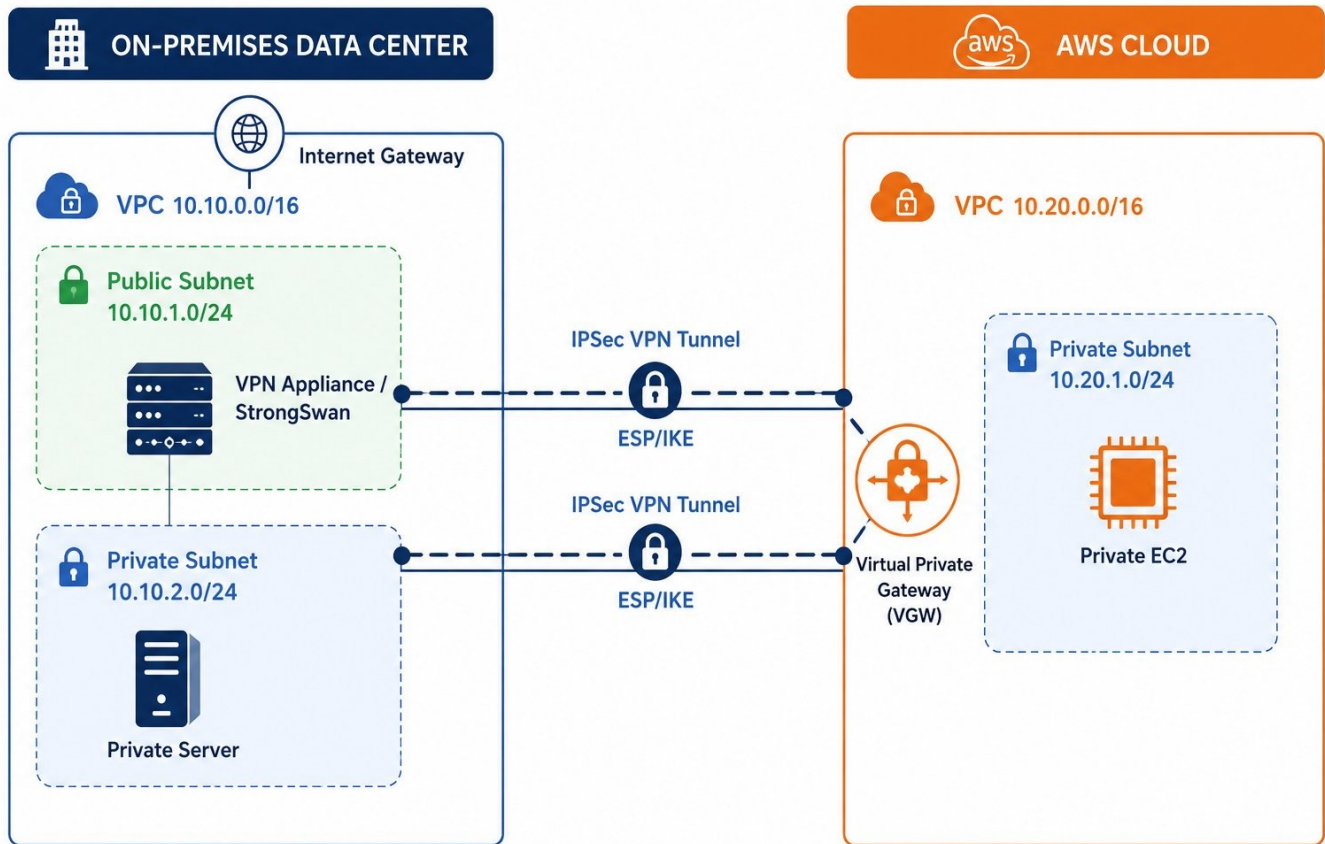


Figure: AWS Site-to-Site VPN Architecture — showing On-Premises VPC (left), Public Internet / IPSec tunnel (center), and AWS Cloud VPC (right) with all networking components, IP addresses, route tables, and security group rules.

12. Conclusion

This documentation walked through the complete implementation of an AWS Site-to-Site VPN using two VPCs — one simulating an On-Premises corporate network and one representing the AWS Cloud. Every component — from the VPC infrastructure, through the VPN appliance setup, to the strongSwan IPSec configuration, route tables, and end-to-end packet flow — was covered in detail.

The successful ping from 10.10.2.47 to 10.20.1.177 represents one of the most fundamental achievements in hybrid cloud networking: two isolated private networks communicating securely across the public internet, with every packet encrypted end-to-end using industry-standard AES-128 encryption.

This project mirrors exactly what Fortune 500 companies do when they first connect their data centers to AWS, and the skills developed here are directly applicable to real-world enterprise cloud architecture, cloud migration projects, and hybrid cloud operations.

Project Summary

OnPrem VPC CIDR: 10.10.0.0/16
AWS Cloud VPC CIDR: 10.20.0.0/16
VPN Protocol: IKEv1 / IPSec (AES-128 + SHA-1 + DH Group 2)
Software VPN: strongSwan on Amazon Linux 2
VPN Appliance EIP: 13.126.73.216
AWS Tunnel Endpoint: 15.207.7.27
VGW ID: vgw-028a9ed0c1722a0f8
CGW ID: cgw-0343d289f5518124a
VPN Connection ID: vpn-0c196248dd7221d3b
End-to-End Test: ping from 10.10.2.47 to 10.20.1.177 — SUCCESS